بسم الله الرحمن الرحیم



موضوع: تکلیف سری اول هوش مصنوعی مقدماتی

نام و نام خانوادگی: محمدمهدی آزادی

شماره دانشجویی: 40214623

نام استاد: استاد تشنه لب

تکلیف درس هوش مصنوعی مقدماتی

1404-1405

با کلیک روی هر عنوان، به بخش مربوطه هدایت می‌شوید.

فهرست

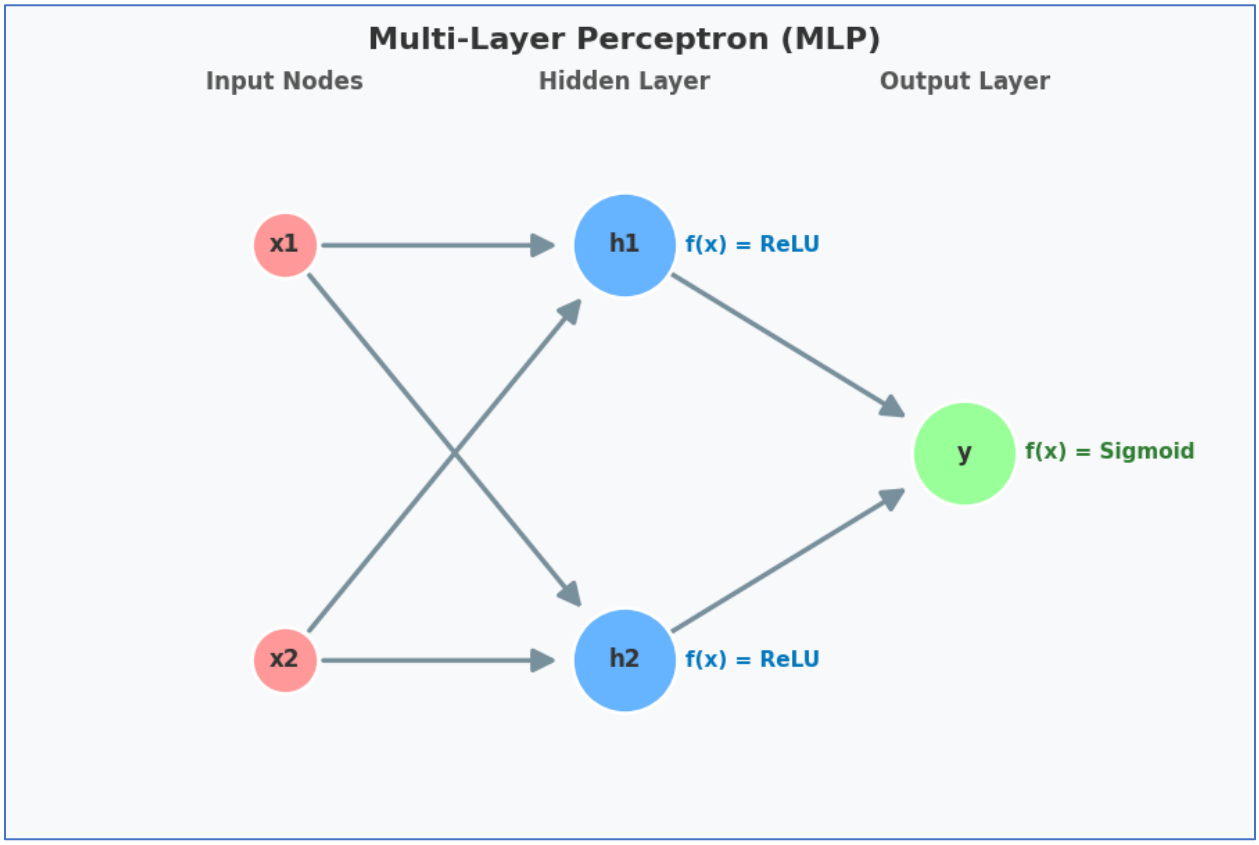
سوال ۱ ..... ۱
سوال ۲ ..... ۱۰
سوال ۳ ..... ۱۴

سوال اول:

برای حل مسئله XOR با دو ورودی، مینیمم ساختار لازم برای یک شبکه عصبی (یک لایه پنهان با ۲ نورون و یک خروجی، یعنی معماری 2-2-1) را طراحی کنید. پس از رسم شکل شبکه و تعیین وزنهای نهایی، فرایند آموزش و همگرایی شبکه را به‌صورت مرحله‌ای شرح دهید.»

(راهنمایی : تابع XOR به صورت خطی تفکیک‌پذیر نیست؛ بنابراین پیاده‌سازی آن با معماری ۲-۱-۱ و توابع فعالساز استاندارد خطی یا پله‌ای ممکن نیست و به همین دلیل در این مسئله به حداقل دو نورون در لایه پنهان نیاز دارید.)

شبکه ما به شکل زیر در می آید:



برای **آموزش واقعی** شبکه با استفاده از الگوریتم پس‌انتشار خطا (Backpropagation) و گرادیان کاهشی، استفاده از توابعی که شما انتخاب کردید بسیار بهتر از تابع پله‌ای است.

دلایل آن:

۱. **لایه پنهان (ReLU)**: تابع ReLU غیرخطی بودنِ لازم برای حل مسئله XOR را فراهم می‌کند و همچنین مشکل محو شدن گرادیان (Vanishing Gradient) را در طول آموزش ندارد.

۲. **لایه خروجی (Sigmoid)**: از آنجا که مسئله XOR یک دسته‌بندی باینری (خروجی ۰ یا ۱) است، تابع سیگموید بهترین انتخاب برای لایه خروجی است زیرا عدد نهایی را بین ۰ و ۱ فشرده می‌کند که می‌توان آن را به عنوان احتمال در نظر گرفت (مثلاً اگر بزرگتر از ۰.۵ بود یعنی ۱ و در غیر این صورت یعنی ۰).

۱. تعریف ساختار و متغیرها

- **ورودی** (X): بردار 2×1
- **لایه پنهان** (L_1): وزن‌ها $W^{[1]}$(ماتریس 2×2)، بایاس $b^{[1]}$(بردار 2×1)، تابع فعال‌سازی ReLU
- **لایه خروجی** (L_2): وزن‌ها $W^{[2]}$(ماتریس 2×1)، بایاس $b^{[2]}$(یک عدد یا 1×1)، تابع فعال‌سازی Sigmoid(σ)
- **خروجی واقعی** (Y): مقدار مطلوب (۰ یا ۱)

۲. مسیر پیش‌رو (Forward Propagation)

ابتدا مقادیر ورودی در شبکه حرکت می‌کنند تا خروجی تخمین‌زده شده ($\hat{Y}$) محاسبه شود:

لایه پنهان:

$$Z^{[1]} = W^{[1]}X + b^{[1]}$$

$$A^{[1]} = \text{ReLU}(Z^{[1]}) = \max(0, Z^{[1]})$$

لایه خروجی:

$$Z^{[2]} = W^{[2]}A^{[1]} + b^{[2]}$$

$$A^{[2]} = \sigma(Z^{[2]}) = \frac{1}{1 + e^{-Z^{[2]}}}$$

(در اینجا $\hat{Y} = A^{[2]}$ خروجی نهایی شبکه است).

۳. تابع هزینه (Cost Function)

برای سادگی در آموزش، از تابع خطای میانگین مربعات (MSE) برای یک نمونه داده استفاده می‌کنیم:

$$E = \frac{1}{2}(A^{[2]} - Y)^2$$

۴. پس‌انتشار خطا (Backward Propagation)

حال باید مشتق تابع خطا نسبت به تک‌تک وزن‌ها و بایاس‌ها محاسبه شود (با استفاده از قاعده زنجیره‌ای):

محاسبات لایه خروجی (L_2):

مشتق خطا نسبت به خروجی:

$$dA^{[2]} = \frac{\partial E}{\partial A^{[2]}} = (A^{[2]} - Y)$$

مشتق نسبت به ورودی خطی لایه خروجی (با توجه به اینکه مشتق سیگموید برابر $\sigma(z)(1 - \sigma(z))$است):

$$dZ^{[2]} = dA^{[2]} \cdot \sigma'(Z^{[2]}) = (A^{[2]} - Y) \cdot A^{[2]}(1 - A^{[2]})$$

گرادیان‌های لایه خروجی:

$$dW^{[2]} = dZ^{[2]} \cdot A^{[1]T}$$

$$db^{[2]} = dZ^{[2]}$$

محاسبات لایه پنهان (L_1):

مشتق خطای منتقل شده به لایه پنهان:

$$dA^{[1]} = W^{[2]T} \cdot dZ^{[2]}$$

مشتق نسبت به ورودی خطی لایه پنهان (مشتق ReLU برابر ۱ برای مقادیر مثبت و ۰ برای مقادیر منفی/صفر است):

$$dZ^{[1]} = dA^{[1]} * \text{ReLU}'(Z^{[1]})$$

*(نکته: علامت * ضرب درایه به درایه (ضرب هادامارد) است و* $\text{ReLU}'(z)$ *اگر* $z > 0$ *باشد ۱ و در غیر این صورت ۰ است).*

گرادیان‌های لایه پنهان:

$$dW^{[1]} = dZ^{[1]} \cdot X^T$$

$$db^{[1]} = dZ^{[1]}$$

۵. به‌روزرسانی وزن‌ها (Weight Update)

در نهایت، با استفاده از الگوریتم گرادیان کاهشی (Gradient Descent) و نرخ یادگیری α، وزن‌ها و بایاس‌ها به‌روز می‌شوند:

$$W^{[2]} \leftarrow W^{[2]} - \alpha \cdot dW^{[2]}$$

$$b^{[2]} \leftarrow b^{[2]} - \alpha \cdot db^{[2]}$$

$$W^{[1]} \leftarrow W^{[1]} - \alpha \cdot dW^{[1]}$$

$$b^{[1]} \leftarrow b^{[1]} - \alpha \cdot db^{[1]}$$

این مراحل برای تمام نمونه‌های جدول درستی XOR تکرار می‌شوند تا زمانی که خطای شبکه (E) به کمترین مقدار ممکن برسد.

حال مثال عددی از مسیر رفت و برگشت می‌زنیم تا قابل درک‌تر باشد:

برای انجام یک مثال عددی، مقادیر اولیه را به صورت تصادفی فرض می‌کنیم. فرض کنید می‌خواهیم شبکه را برای یکی از حالت‌های XOR، مثلاً ورودی $X = [1,0]^T$ که خروجی واقعی آن $Y = 1$ است، یک مرحله آموزش دهیم.

۱. مقادیر اولیه (فرض شده)

- **ورودی:** $X = \begin{bmatrix}1\\0\end{bmatrix}$، **خروجی مطلوب:** $Y = 1$
- **وزن‌ها و بایاس لایه پنهان** (L_1):

$$W^{[1]} = \begin{bmatrix}0.5 & -0.5\\0.2 & 0.8\end{bmatrix}, b^{[1]} = \begin{bmatrix}-0.1\\0.1\end{bmatrix}$$

- **وزن‌ها و بایاس لایه خروجی** (L_2):

$$W^{[2]} = [0.6 \quad -0.4], b^{[2]} = [0.2]$$

- **نرخ یادگیری:** $\alpha = 0.5$

---

۲. مسیر پیش‌رو (Forward Propagation)

لایه پنهان:

$$Z^{[1]} = W^{[1]}X + b^{[1]} = \begin{bmatrix}0.5 & -0.5\\0.2 & 0.8\end{bmatrix}\begin{bmatrix}1\\0\end{bmatrix} + \begin{bmatrix}-0.1\\0.1\end{bmatrix} = \begin{bmatrix}0.5\\0.2\end{bmatrix} + \begin{bmatrix}-0.1\\0.1\end{bmatrix}$$
$$= \begin{bmatrix}0.4\\0.3\end{bmatrix}$$

$$A^{[1]} = \text{ReLU}\left(\begin{bmatrix}0.4\\0.3\end{bmatrix}\right) = \begin{bmatrix}0.4\\0.3\end{bmatrix}$$

(چون هر دو مثبت هستند، مقدارشان تغییر نمی‌کند).

لایه خروجی:

$$Z^{[2]} = W^{[2]}A^{[1]} + b^{[2]} = [0.6 \quad -0.4]\begin{bmatrix}0.4\\0.3\end{bmatrix} + 0.2$$
$$= (0.24 - 0.12) + 0.2 = 0.32$$

$$A^{[2]} = \sigma(0.32) = \frac{1}{1+e^{-0.32}} \approx 0.579$$

محاسبه خطا (صرفاً جهت اطلاع):

$$E = \frac{1}{2}(A^{[2]} - Y)^2 = \frac{1}{2}(0.579 - 1)^2 \approx 0.088$$

---

۳. پس‌انتشار خطا (Backward Propagation)

لایه خروجی (L_2):

$$dA^{[2]} = A^{[2]} - Y = 0.579 - 1 = -0.421$$

$$dZ^{[2]} = dA^{[2]} \cdot (A^{[2]}(1 - A^{[2]})) = -0.421 \cdot (0.579 \times 0.421)$$
$$\approx -0.102$$

$$dW^{[2]} = dZ^{[2]} \cdot A^{[1]T} = -0.102 \cdot [0.4 \quad 0.3] = [-0.0408 \quad -0.0306]$$

$$db^{[2]} = dZ^{[2]} = -0.102$$

لایه پنهان (L_1):

$$dA^{[1]} = W^{[2]T} \cdot dZ^{[2]} = \begin{bmatrix} 0.6 \\ -0.4 \end{bmatrix} \cdot (-0.102) = \begin{bmatrix} -0.0612 \\ 0.0408 \end{bmatrix}$$

چون در مرحله پیش‌رو مقادیر $Z^{[1]}$(یعنی 0.4و 0.3) هر دو مثبت بودند، مشتق ReLU برای هر دو برابر 1است:

$$dZ^{[1]} = dA^{[1]} * \begin{bmatrix} 1 \\ 1 \end{bmatrix} = \begin{bmatrix} -0.0612 \\ 0.0408 \end{bmatrix}$$

$$dW^{[1]} = dZ^{[1]} \cdot X^T = \begin{bmatrix} -0.0612 \\ 0.0408 \end{bmatrix} \cdot [1 \quad 0] = \begin{bmatrix} -0.0612 & 0 \\ 0.0408 & 0 \end{bmatrix}$$

$$db^{[1]} = dZ^{[1]} = \begin{bmatrix} -0.0612 \\ 0.0408 \end{bmatrix}$$

۴. به‌روزرسانی پارامترها (Weight Update)

با فرمول $W_{new} = W_{old} - \alpha \cdot dW$:

لایه خروجی:

$$W_{new}^{[2]} = [0.6 \quad -0.4] - 0.5[-0.0408 \quad -0.0306]$$
$$= [0.6204 \quad -0.3847]$$

$$b_{new}^{[2]} = 0.2 - 0.5(-0.102) = 0.251$$

لایه پنهان:

$$W_{new}^{[1]} = \begin{bmatrix} 0.5 & -0.5 \\ 0.2 & 0.8 \end{bmatrix} - 0.5 \begin{bmatrix} -0.0612 & 0 \\ 0.0408 & 0 \end{bmatrix} = \begin{bmatrix} 0.5306 & -0.5 \\ 0.1796 & 0.8 \end{bmatrix}$$

$$b_{new}^{[1]} = \begin{bmatrix} -0.1 \\ 0.1 \end{bmatrix} - 0.5 \begin{bmatrix} -0.0612 \\ 0.0408 \end{bmatrix} = \begin{bmatrix} -0.0694 \\ 0.0796 \end{bmatrix}$$

این فرآیند نشان داد که شبکه چگونه مقادیر وزن‌ها را برای نزدیک شدن خروجی به عدد ۱ (مقدار مطلوب) تنظیم کرد.

سوال دوم:

اگر تابع هزینه به صورت زیر تعریف شده باشد،روند آموزش شبکه عصبی MLP را با الگوریتم گرادیان نزولی بیان نمایید.

$$E=\frac{1}{2}\sum_{k=1}^{K}(e_k)^2+\sum_{j=1}^{n_1}(w_j^1)^2+\sum_{q=1}^{n_2}(w_q^2)^2$$

برای بیان روند آموزش با این تابع هزینه، باید قوانین به‌روزرسانی وزن‌ها را در الگوریتم گرادیان نزولی (Gradient Descent) استخراج کنیم.

تابع هزینه داده شده در تصویر به این شکل است:

$$E=\frac{1}{2}\sum_{k=1}^{K}(e_k)^2+\sum_{j=1}^{n_1}(w_j^1)^2+\sum_{q=1}^{n_2}(w_q^2)^2$$

این تابع از دو بخش اصلی تشکیل شده است:

۱. **خطای استاندارد (MSE)**: $E_{data}=\frac{1}{2}\sum_{k=1}^{K}(e_k)^2$ که در آن $e_k=t_k-y_k$ (تفاضل خروجی مطلوب t و خروجی واقعی شبکه y و این همان خطایی است که در حالت عادی مسیر برگشت محاسبه می‌کنیم).

۲. **جریمه وزن‌ها (L2 Regularization)**: مجموع مربعات وزن‌های لایه اول (w^1) و لایه دوم (w^2). این بخش باعث ایجاد اثری به نام **Weight Decay** (**کاهش وزن**) می‌شود.

قانون کلی به‌روزرسانی وزن‌ها در گرادیان نزولی به شکل زیر است:

$$w_{new} = w_{old} - \eta \frac{\partial E}{\partial w}$$

که در آن η نرخ یادگیری (Learning Rate) است.

۱. به‌روزرسانی وزن‌های لایه خروجی (W^2)

فرض کنید w_{kj}^2 وزنی باشد که نورون پنهان jرا به نورون خروجی kمتصل می‌کند. باید مشتق کل تابع Eرا نسبت به این وزن محاسبه کنیم:

$$\frac{\partial E}{\partial w_{kj}^2} = \frac{\partial E_{data}}{\partial w_{kj}^2} + \frac{\partial}{\partial w_{kj}^2}\left(\sum_{q=1}^{n_2}(w_q^2)^2\right)$$

- **بخش اول (مشتق خطای داده):** طبق قوانین استاندارد پس‌انتشار (Backpropagation)، این مشتق برابر است با:

$$\frac{\partial E_{data}}{\partial w_{kj}^2} = -(t_k - y_k) \cdot f'(net_k) \cdot h_j = -\delta_k \cdot h_j$$

(در اینجا h_jخروجی نورون پنهان j، f'مشتق تابع فعال‌ساز لایه خروجی، و $\delta_k = (t_k - y_k)f'(net_k)$سیگنال خطای لایه خروجی است).

- **بخش دوم (مشتق جریمه):** مشتق مجموع مربعات وزن‌ها نسبت به یک وزن خاص w_{kj}^2برابر است با:

$$\frac{\partial}{\partial w_{kj}^2}\left(\sum_{q=1}^{n_2}(w_q^2)^2\right) = 2w_{kj}^2$$

بنابراین، گرادیان کل برای لایه خروجی برابر است با:

$$\frac{\partial E}{\partial w_{kj}^2} = -\delta_k h_j + 2w_{kj}^2$$

فرمول نهایی به‌روزرسانی لایه خروجی:

$$w_{kj}^2(new) = w_{kj}^2(old) - \eta\left(-\delta_k h_j + 2w_{kj}^2(old)\right)$$

$$w_{kj}^2(new) = w_{kj}^2(old)\cdot(1-2\eta) + \eta\delta_k h_j$$

---

۲. به‌روزرسانی وزن‌های لایه پنهان (W^1)

فرض کنید w_{ji}^1 وزنی باشد که ورودی i را به نورون پنهان j متصل می‌کند. مشابه حالت قبل، مشتق می‌گیریم:

$$\frac{\partial E}{\partial w_{ji}^1} = \frac{\partial E_{data}}{\partial w_{ji}^1} + \frac{\partial}{\partial w_{ji}^1}\left(\sum_{q=1}^{n_1}\left(w_q^2\right)^2\right)$$

- **بخش اول (مشتق خطای داده):** با استفاده از قاعده زنجیره‌ای برای لایه پنهان:

$$\frac{\partial E_{data}}{\partial w_{ji}^{1}} = -\left(\sum_{k=1}^{K} \delta_k \, w_{kj}^{2}\right) \cdot f'(net_j) \cdot x_i = -\delta_j \cdot x_i$$

(در اینجا δ_j سیگنال خطای لایه پنهان است).

- **بخش دوم (مشتق جریمه):**

$$\frac{\partial}{\partial w_{ji}^{1}} \left(\sum_{q=1}^{n_1} \left(w_q^2\right)^2\right) = 2w_{ji}^{1}$$

بنابراین، گرادیان کل برای لایه پنهان برابر است با:

$$\frac{\partial E}{\partial w_{ji}^{1}} = -\delta_j x_i + 2w_{ji}^{1}$$

فرمول نهایی به‌روزرسانی لایه پنهان:

$$w_{ji}^{1}(new) = w_{ji}^{1}(old) - \eta\left(-\delta_j x_i + 2w_{ji}^{1}(old)\right)$$

$$w_{ji}^{1}(new) = w_{ji}^{1}(old) \cdot (1 - 2\eta) + \eta\delta_j x_i$$

نتیجه‌گیری

اضافه شدن جملات توانی وزن‌ها به تابع هزینه باعث می‌شود که در هر مرحله از به‌روزرسانی، وزن قبلی در ضریب $(1-2\eta)$ضرب شود (از آنجا که معمولاً ηکوچک است، این ضریب کمی کمتر از ۱ خواهد بود). این دقیقاً همان مفهوم **کاهش وزن** (**Weight Decay**) است که باعث می‌شود شبکه همواره تمایل داشته باشد وزن‌های خود را کوچک نگه دارد تا از بیش‌برازش (Overfitting) جلوگیری کند. بایاس‌ها (Bias) معمولاً شامل این جریمه نمی‌شوند و به همان روش استاندارد (بدون ضریب کاهنده) بروزرسانی می‌شوند.

سوال سوم:

برای تقریب هر یک از توابع زیر یک شبکه عصبی MLP طراحی نمایید. با رسم نمودار خروجی شبکه و نمودار خطا در مورد بازه متغیر های ورودی بحث نمایید.

$$f(x_1,x_2)=\max\left\{e^{-x_1^2},e^{-2x_2^2},2e^{-0.5\left(x_1^2+x_2^2\right)}\right\}$$

$$f(x,y)=0.5+0.1x^2\cos(y+3)+0.4xye^{1-y^2}$$

طراحی و آموزش چنین شبکه‌ای (پیدا کردن وزن‌ها و بایاس‌ها) نیازمند استفاده از الگوریتم‌های بهینه‌سازی (مانند گرادیان نزولی) طی هزاران Epoch روی مجموعه‌ای از داده‌های تولید شده است.

بر اساس **قضیه تقریب جامع (Universal Approximation Theorem)**، یک شبکه MLP با حداقل یک لایه پنهان، تعداد کافی نورون و یک تابع فعال‌ساز غیرخطی (مانند ReLU، Tanh یا Sigmoid) می‌تواند هر تابع پیوسته‌ای را تقریب بزند.

۱. طراحی ساختار شبکه (معماری MLP)

برای هر دو تابع $f(x_1,x_2)$ و $f(x,y)$ ساختار زیر مناسب است:

- **لایه ورودی:** ۲ نورون (برای دو متغیر ورودی).
- **لایه‌های پنهان:** برای توابع پیچیده، استفاده از ۲ لایه پنهان با ۲۰ تا ۶۴ نورون در هر لایه و تابع فعال‌ساز ReLU یا Tanh عملکرد بسیار خوبی دارد (مثلاً دو لایه ۶۴ نورونی).
- **لایه خروجی:** ۱ نورون با تابع فعال‌ساز خطی (Linear)، زیرا خروجی ما یک مقدار پیوسته (رگرسیون) است، نه دسته‌بندی.
- **تابع هزینه:** میانگین مربعات خطا (MSE).

برای انتخاب بازه ورودی مناسب جهت آموزش شبکه عصبی، باید به رفتار ریاضی هر تابع توجه کنیم. شبکه‌های عصبی در بازه‌ای که آموزش دیده‌اند (درون‌یابی یا Interpolation) عملکرد خوبی دارند، اما در خارج از آن (برون‌یابی یا Extrapolation) معمولاً دچار خطای شدید می‌شوند.

در اینجا رفتار هر دو تابع و بازه‌های پیشنهادی را بررسی می‌کنیم:

تابع اول:

$$f(x_1, x_2) = \max\left(e^{-x_1^2}, e^{-2x_2^2}, 2e^{-0.5(x_1^2+x_2^2)}\right)$$

تحلیل رفتار تابع:

- هر سه عبارت داخل تابع max، توابع نمایی-گاوسی (شبیه زنگوله) هستند که بیشترین مقدار خود را در مبدأ یعنی نقطه $(0, 0)$دارند.
- با دور شدن از مبدأ (افزایش قدر مطلق x_1و x_2)، مقدار این توابع به سرعت به سمت صفر میل می‌کند.
- به عنوان مثال، برای عبارت $e^{-x_1^2}$وقتی $x_1 = 3$باشد، مقدار تابع $e^{-9} \approx 0.0001$خواهد بود که عملاً صفر است.

بازه پیشنهادی:

بازه $[-3,3]$ **یا** $[-4,4]$برای هر دو متغیر x_1 و x_2بسیار ایده‌آل است.

در این بازه، شبکه عصبی تمام افت و خیزهای مهم، قله‌ی تابع (در مبدأ) و مرزهای شکستگی ناشی از تابع maxرا می‌بیند. خارج از این بازه، تابع تقریباً یک صفحه مسطح با مقدار صفر است و آموزش شبکه روی بازه بزرگتر (مثلاً $[-10,10]$) باعث می‌شود شبکه زمان زیادی را صرف یادگیری یک ناحیه صاف و بدون اطلاعات کند که کارایی را کاهش می‌دهد.

تابع دوم:

$$f(x,y) = 0.5 + 0.1x^2\cos(y+3) + 0.4xye^{1-y^2}$$

تحلیل رفتار تابع:

این تابع رفتار پیچیده‌تری دارد زیرا شامل نوسان، رشد چندجمله‌ای و افت نمایی است:

1. **عبارت** $0.1x^2\cos(y+3)$: در راستای محور xرشد سهمی‌وار (درجه ۲) دارد و در راستای محور yبه صورت کسینوسی نوسان می‌کند. دوره تناوب کسینوس $2\pi \approx 6.28$است.
2. **عبارت** $0.4xye^{1-y^2}$: در راستای محور xخطی است، اما در راستای محور yبه دلیل وجود e^{-y^2}، پس از یک اوج و حضیض در حوالی $y = \pm 1$، به سرعت به سمت صفر میل می‌کند.

بازه پیشنهادی:

- برای متغیر y: بازه‌ای مانند $[-4,4]$ **یا** $[-5,5]$مناسب است. این بازه تضمین می‌کند که شبکه حداقل یک تناوب کامل از رفتار کسینوسی را ببیند و همچنین ناحیه‌ای که بخش نمایی (e^{1-y^2}) در آن تأثیرگذار است را کاملاً پوشش دهد.
- برای متغیر x: چون تابع در راستای xتا بی‌نهایت رشد می‌کند (x^2)، هر بازه‌ای که انتخاب کنیم، شبکه فقط همان قسمت را یاد می‌گیرد. بازه $[-3,3]$ **تا** $[-5,5]$برای نشان دادن انحنای سهمی‌وار تابع کافی است.

نکته مهم در مورد تابع دوم:

اگر شبکه را روی بازه $x \in [-5,5]$ آموزش دهیم و سپس بخواهیم مقدار تابع را برای مقداری مثل $x = 10$ پیش‌بینی کنیم، شبکه عصبی به احتمال زیاد خطای فاحشی خواهد داشت. شبکه‌های MLP توابع تناوبی (مثل کسینوس) و توابع با رشد نامحدود (مثل x^2) را در خارج از داده‌های آموزشی به درستی تعمیم نمی‌دهند. بنابراین، بازه انتخاب شده دقیقاً همان محدوده‌ای است که شبکه برای استفاده در دنیای واقعی در آن معتبر خواهد بود.

ساخت و آموزش شبکه عصبی:

```python
import numpy as np
import matplotlib.pyplot as plt
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense
from tensorflow.keras.optimizers import Adam
from mpl_toolkits.mplot3d import Axes3D

def target_function(function_number, x, y):
    if function_number == 1:
        return 0.5 + 0.1 * (x**2) * np.cos(y + 3) + 0.4 * x * y * np.exp(1 - y**2)
    elif function_number == 2:
        term1 = np.exp(-(x**2))
        term2 = np.exp(-2*(y**2))
        term3 = 2 * np.exp(-0.5 * (x**2 + y**2))
        return np.maximum.reduce([term1, term2, term3])

range_val = 3.0
x = np.linspace(-range_val, range_val, 50)
y = np.linspace(-range_val, range_val, 50)
X, Y = np.meshgrid(x, y)

x_flat = X.flatten()
y_flat = Y.flatten()
inputs = np.column_stack((x_flat, y_flat))
targets = target_function(1, x_flat, y_flat)

model = Sequential([
    Dense(64, activation='relu', input_dim=2),
    Dense(64, activation='relu'),
    Dense(1, activation='linear')
])

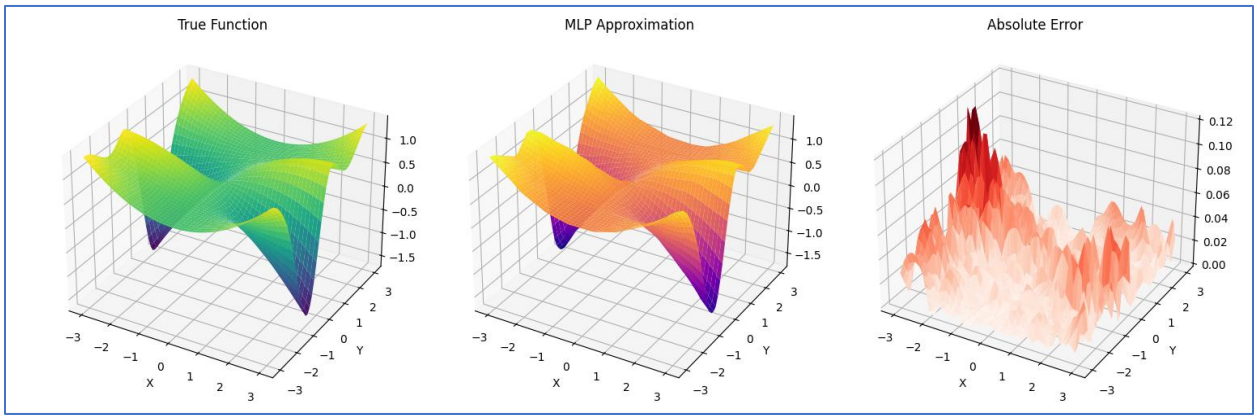
model.compile(optimizer=Adam(learning_rate=0.01), loss='mse')

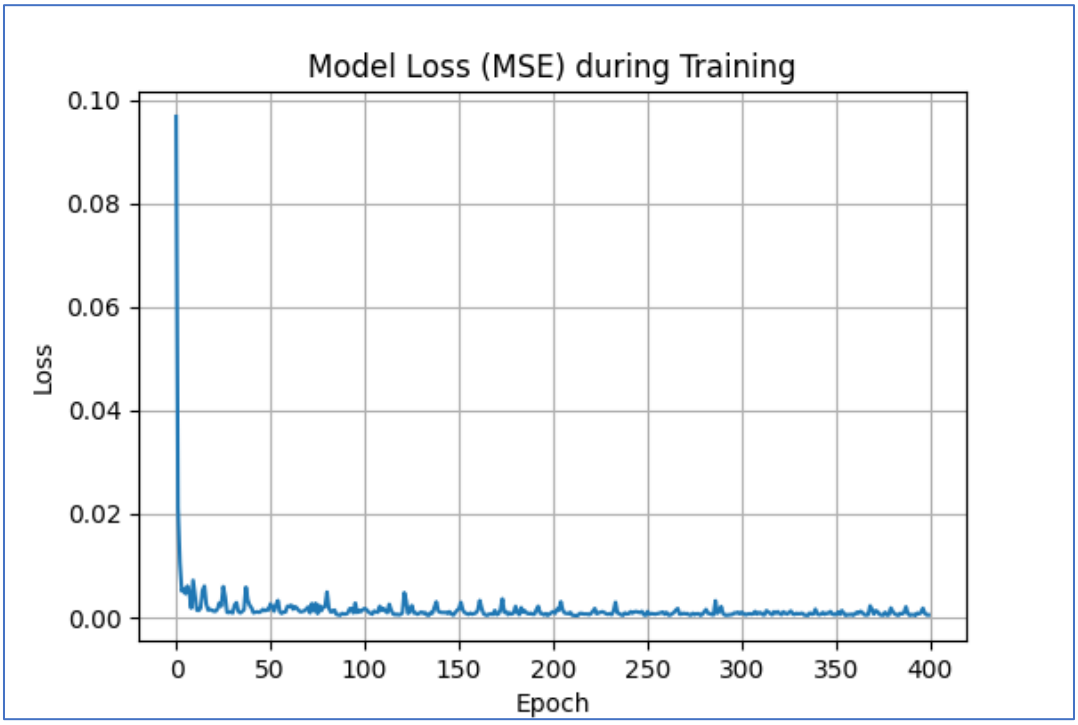
print("learning progress...")
history = model.fit(inputs, targets, epochs=400, verbose=0, batch_size=32)
print("End of learning")

predictions = model.predict(inputs).flatten()
predictions_grid = predictions.reshape(X.shape)
```

```
41
42    error_grid = np.abs(targets.reshape(X.shape) - predictions_grid)
43
44    fig = plt.figure(figsize=(18, 5))
45
46    ax1 = fig.add_subplot(131, projection='3d')
47    ax1.plot_surface(X, Y, targets.reshape(X.shape), cmap='viridis')
48    ax1.set_title('True Function')
49    ax1.set_xlabel('X') ; ax1.set_ylabel('Y')
50
51    ax2 = fig.add_subplot(132, projection='3d')
52    ax2.plot_surface(X, Y, predictions_grid, cmap='plasma')
53    ax2.set_title('MLP Approximation')
54    ax2.set_xlabel('X') ; ax2.set_ylabel('Y')
55
56    ax3 = fig.add_subplot(133, projection='3d')
57    ax3.plot_surface(X, Y, error_grid, cmap='Reds')
58    ax3.set_title('Absolute Error')
59    ax3.set_xlabel('X') ; ax3.set_ylabel('Y')
60
61    plt.tight_layout()
62    plt.show()
63
64    plt.figure(figsize=(6, 4))
65    plt.plot(history.history['loss'])
66    plt.title('Model Loss (MSE) during Training')
67    plt.ylabel('Loss')
68    plt.xlabel('Epoch')
69    plt.grid(True)
70    plt.show()
71
```

نمودار برای function 1 :





نمودار برای function 2 :

خط 25 را هم به مقدار تغییر می دهیم:

```
targets = target_function(2, x_flat, y_flat)
```

